

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术
实验项目编号 1 实验名称 处理器基本功能部件设计——寄存器堆和算术逻辑单元

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与仿真波形的截图及说明(比如 Verilog HDL 关键代码段 {包含注释} 及其对应的逻辑电路结构图 {自行画图, 推荐用 PPT 画逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX. 中}、相应信号的仿真波形和信号变化的说明等)

- 寄存器堆设计。首先, 我们回顾一下基本代码, 阐述其中重要思想和逻辑, 然后画出逻辑图, 而后观察波形。1. 定义寄存器堆: 同样数据位宽的数组, 以实现存储 (memory)

```
1 reg [`DATA_WIDTH-1:0] reg_dump [`DATA_WIDTH-1:0]; //定义寄存器堆
```

2. 选择对应地址的寄存器堆(除了 0 寄存器之外): 直接用 `reg_dump[raddr]` 就可以选择; 因为是异步读取, 所以是组合逻辑

```
1 assign rdata1 = {`DATA_WIDTH{(raddr1 != 0)}} & reg_dump[raddr1]; //异步读取 (地址是0直接读0)
2 assign rdata2 = {`DATA_WIDTH{(raddr2 != 0)}} & reg_dump[raddr2];
```

3. 关于 0 寄存器: 我不管里面存的是什么, 只要每次我读出来是 0, 那么就可以当成里面是 0。所以当 `raddr1==0` 的时候, `rdata` 就是 0 即可

4. 关于同步写: 时钟上升沿的时候, 如果写使能有效, 那么在不是 0 地址的地方, 写入数据即可。注意使用非阻塞赋值, 因为是时序逻辑

```
1 always @(posedge clk) begin
2   if (wen) begin
3     if (waddr != `ADDR_WIDTH'b0)begin
4       reg_dump[waddr]<=wdata; //非阻塞赋值同步写入
5     end
6   end
7 end
```

逻辑如下:

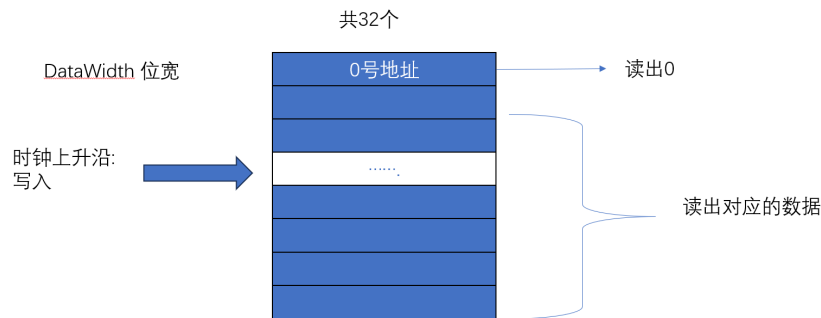


图 1: 寄存器堆的逻辑图

接着观察波形:

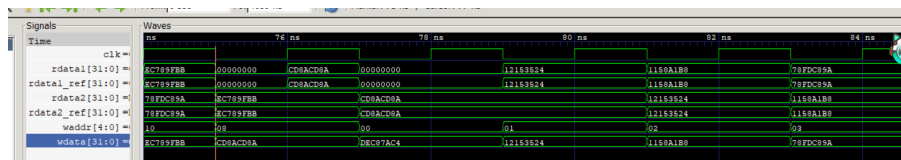


图 2: 寄存器堆波形图

我们可以看到,寄存器的 ref 输出和我的输出完全一样,说明没有问题。

- **ALU 设计。** 首先,这次的 ALU 有几个难点: 第一个难点是只能使用一个加法器。这个引起了我的思考: 怎么样才能够实现只用一个加法器呢?

减法:不能直接调用 $A + B + 1$, 因为这样会和加法中的 $A + B$ 综合成使用两个加法器。

所以,我想,加法器应该是不变的,应该改变的是输入的 B 和 carry_in (进位项),这告诉我们:应该有一个加法器的模块封装好,会更加方便。

第二个难点是如何判断进位和借位? 可以想到,如果有进位的话,加法会进位出一个 1; 如果有借位的话,减法会借位减出一个 1。这样一来,如果算出来之后的第 33 位是 1,那么就是出现了 carryout 的状态。

综上,我抽象了一个加法模块,其中输入和输出都是 33 位的,从而便于判断 carryout; 同时设置了 carryin, 这样便于减法的加 1。

代码如下:

```
1 module adder33 (  
2   input  [`DATA_WIDTH:0] A,  
3   input  [`DATA_WIDTH:0] B,  
4   input  carry_in,  
5   output [`DATA_WIDTH-1:0] add_result,  
6   output carryout  
7 );  
8 assign {carryout,add_result} = A + B + {32'b0,carry_in};  
    //carryout是第33位, add_result是后面的共32位数字
```

那么此时,加法就是输入 A 、 B , carryin 是 0, 即可得到; 减法输入 A , 和 B 的取反, carryin 是 1, 即可得到减法。而此时,第 33 位输出的就是借位/进位情况。即:

```

1 assign B_change = (ALUop==`ADD)?B:~B;
2 //减法就是B取反, 加法B就是原来的值
3 assign c_in = (ALUop == `ADD)? 0:1;
4 //输入的c_in, 减法就是1, 加法就是0
5 adder33 u_adder(.A({A[`DATA_WIDTH-1],A}),.B({B_change
6     [`DATA_WIDTH-1],B_change}),.carry_in(c_in), .add_result(add_sub_result),.carryout(CarryOut));
7 //实例化加法器逻辑块A和B需要扩展符号位

```

那么这个时候, 还有第三个问题: 如何实现 overflow 的问题? overflow 是正数 + 正数得到负数, 负数加负数得到正数这种符号异常情况(往往是因为算的值的绝对值太大, 取模就改变了原来的值)

综上, 就是如果两个输入加法器的操作数的符号相同, 输出的数字的符号也应该和其相同, 否则溢出。

```

1 assign Overflow = (A[`DATA_WIDTH-1] == B_change[`DATA_WIDTH-1]) && (add_sub_result[`DATA_WIDTH-1]
2     != A[`DATA_WIDTH-1]);

```

而这个时候, slt 也可以迎刃而解了: add_sub_result 如果在不是 ADD 的时候, 得到的都是减法的结果, 可以通过这个结果的正负得出两个操作数的大小。

一般说来, 如果结果是负, 那么 A 小于 B, slt 应该设成 1; 但是如果发生了溢出, A-B 表征为负数(最高位是 1)的时候, 实际上算出来应该是正数, 所以 A 大于 B。反之亦然。由此得出如下代码:

```

1 assign slt_result = add_sub_result[`DATA_WIDTH-1] ^ Overflow;

```

第四个问题? Result 怎么得到? 加法、减法、slt 都已经得到, 与和或也是直接易得的, 此时, 按照 ALUop 选择即可得出分别的答案:

```

1 assign Result = (ALUop == `ADD)? add_sub_result :
2     (ALUop == `SUB)? add_sub_result :
3     (ALUop == `AND)? A & B :
4     (ALUop == `OR)? A | B :
5     (ALUop == `SLT)? {31'b0,slt_result}:
6     32'b0; //一定最后要有default防止锁存器

```

最后一个小问题就是看输出是不是 0, 用多路选择器即可:

```

1 assign Zero = (Result == 32'b0)? 1:0;

```

二、 实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug, 逻辑仿真和 FPGA 调试过程中的难点等)

ALU 设计问题。第一个问题是语法。比如 wire 定义之后, 后面不能直接接入组合逻辑(不是像 C 语言一样可以直接赋值), 而是要用 assign 单独进行赋值。看来 verilog 的学习之路还任重道远啊。

第二个问题是锁存器逻辑问题, 最开始一直没有赋值 default 是多少, 直到上课助教老师讲了才知道一定所有的分支都必须要有值, 否则会有锁存器。

第三个问题是只能用一个加法器问题, 最开始一直想: 那我如何获得减法呢? 如果我减法算一个 A+B 的取反 +1, 再和加法进行选择, 一定会有两个加法器的吧。那么如果我通过选择 B 的值来仅仅用一个加法器, 加法是 B, 减法是 B 反 +1, 看似没有问题了, 实则仍然会在 B 反加 1 的时候引入一个新的加法器。于是上课的时候, 张

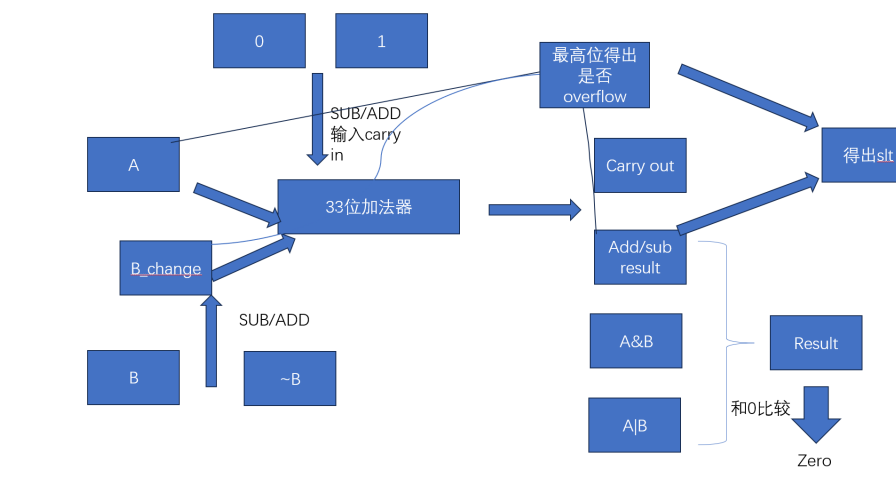


图 3: ALU 流程图

老师讲了 carryin 让我迎刃而解: carryin 设成不同的值就可以解决这个问题了。

第四个人问题是 slt: 这个问题是 LLM 指出我的错误的: 我仅仅考虑了没有 overflow 的情况, 殊不知 overflow 会导致结果的正负号和其应得的产生反转。

第五个人问题是进位和借位: 最开始也一直在想进位和借位的共同点是什么, 甚至差点就加减法进行分类讨论了。后来想到共同点就是更高一位是 0/1 的问题, 毕竟借位和进位都会使第 33 位变成 1。

流程图如下:

三、 对讲义中思考题(如有)的理解和回答

1. 寄存器堆是组合电路还是时序电路?

答: 同步写是时序电路, 异步读是组合逻辑。

2. ALU 的输入为二进制码(比特串), 对应的真值取决于编码规则和定浮点规则, 那么谁来说明这些规则?

答: ALU 不会说明这些规则, 它只进行纯粹的加减法等运算, 它也不知道输入的数字是什么规则。

而谁说明了规则? 指令说明了规则。编译器把高级语言翻译成指令, 指令中说明了数字的类型。比如说, 编译器会在加法后面跟上不同的条件跳转指令。比如比较大小, 编译器如果是看无符号数, 就用 JB (Jump if Below, 看 CF); 如果是看有符号数, 就用 JL (Jump if Less, 看 OF)。

再举一个例子, 如果是整数加法, 编译器生成 ADD 指令。如果是浮点数加法, 编译器生成 FADD (Floating ADD) 指令 (其中浮点数根本不会被送进 ALU)。

ALU 只是个纯粹的物理执行元件, 它对于这些规则是毫不知情的。

3. 有些信号在某些情况下被认为是“无意义/未定义”的。那么被谁来“认为”? 信号在什么情况下是“无意义/未定义”的? 这些输出信号将被谁来“使用”?

答: 先回答第二个问题, 未定义的就是在输出的没有意义的时候, 比如 Overflow 在除了有符号数加减法之外都是没有意义的, 就是未定义的; Carryout 除了无符号数加减法之外也都是没有意义的, 也是未定义的。

那么, 而“认为”的, 就是指令集手册, 它规定了: 在执行某条指令时, 某个信号是垃圾。而遵守这个规则的, 就是编译器和程序员。

既然手册“认为”此时的信号无意义，那么编译器在翻译代码时，就绝对不会在这个时候去使用这个信号。比如，编译器在生成了一个逻辑运算指令后，绝不会紧接着生成一个“根据溢出标志跳转(JO)”的指令，因为它知道这时候的溢出标志是不可信的。

四、 实验所耗时间

在课后，你花费了大约 1.5 小时完成此次实验。